UNIVERSIDAD DE SAN BUENAVENTURA

Optativa II · Ingeniería de Software

EDAD DEL PERRO

Documento de Arquitectura y Diseño

SDD — Diseño de software de Edad del Perro

Basado en IEEE 1016-2009 / ISO/IEC/IEEE 42010 · Código EDP-SAD-001 · Versión 2.0

Integrante	Rol Scrum
Sarah Isabella Peña	Product Owner
Juan Carlos Diaz	Scrum Master
Sebastian Grande Moreno	Developer

Docente: Ing. Jairo Armando Salcedo Aranda

Bogotá D.C., 3 de noviembre de 2026

Control de versiones

Versión	Fecha	Autor	Descripción
1.0	02/11/2026	Equipo Edad del Perro	Versión inicial: calculadora con 3 métodos, etapas de vida, gráfica y chatbot Pepe.
2.0	03/11/2026	Equipo Edad del Perro	Mis perros: guardar, editar, eliminar y actualizar perros; integración con la calculadora; documentación IEEE.

Tabla de contenido

Control de versiones....2
Tabla de contenido....2
1. Introducción....3
2. Arquitectura....3
3. Paquetes....4
4. Modelo de datos....4
5. Clases....5
6. Flujos....5
6.1 Guardar un perro....5
6.2 Abrir un perro en la calculadora....5
7. Decisiones de diseño....6

1. Introducción

Describe la arquitectura, componentes, modelo de datos y decisiones de diseño de Edad del Perro v2.0, incluido el módulo **Mis perros**.

2. Arquitectura

Patrón **MVVM** con UI declarativa (Jetpack Compose). La lógica de negocio está en objetos Kotlin puros (`DogAgeCalculator`, `DogBot`, `DogValidator`, `DogOps`), probados con JUnit sin emulador. Los ViewModels mantienen el estado entre pestañas y `DogStore` persiste los perros.

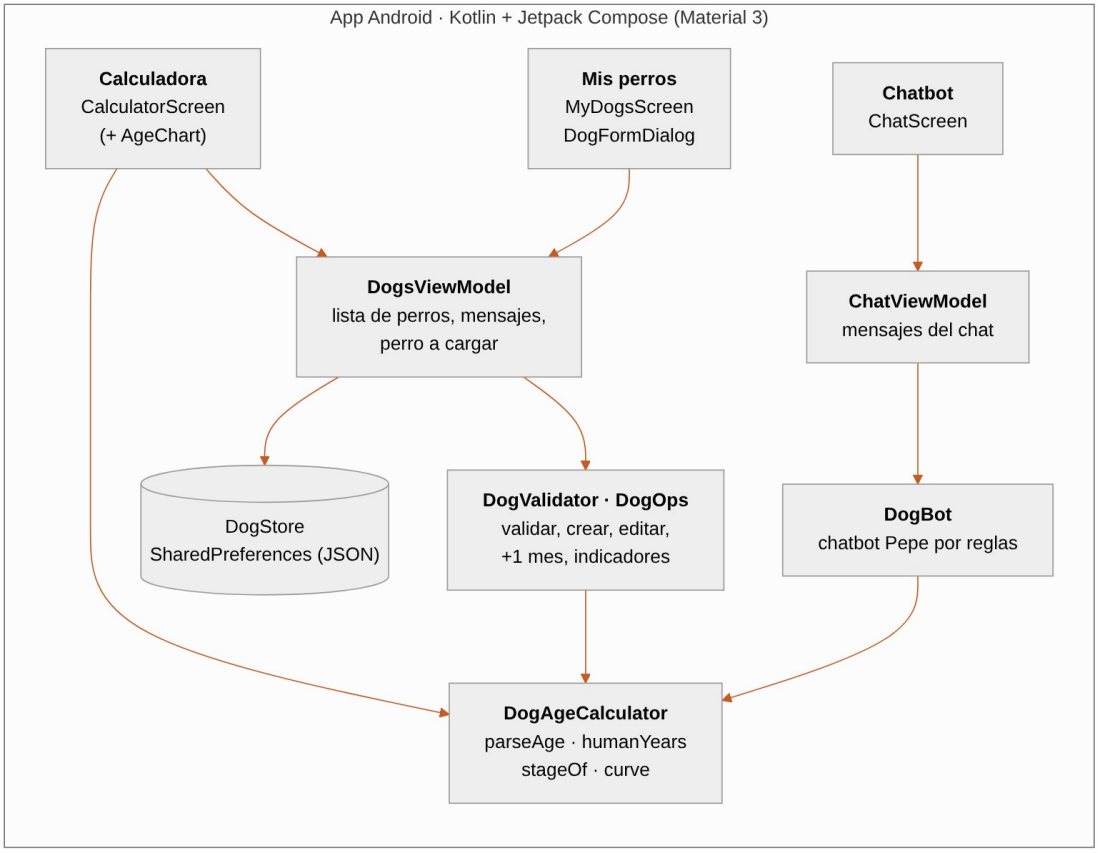


Figura 1. Arquitectura por capas

Capa	Elementos	Responsabilidad
Presentación	CalculatorScreen (AgeChart), MyDogsScreen, DogFormDialog, ChatScreen, Theme	Dibujar el estado y capturar acciones.
Estado	DogsViewModel (AndroidViewModel), ChatViewModel	Estado observable compartido y casos de uso.
Dominio	DogAgeCalculator, DogBot, DogValidator, DogOps	Reglas de negocio puras y probadas.
Datos	Dog, DogSize, CalcMode, LifeStage, DogStore	Modelos y almacenamiento local.

3. Paquetes

```
app/src/main/java/com/optativa/dogage/
├── MainActivity.kt           # 3 pestañas + DogsViewModel compartido
├── data/DogStore.kt          # SharedPreferences (JSON)
├── logic/
│   ├── DogAgeCalculator.kt   # métodos, etapas, validación, curva
│   ├── DogBot.kt             # chatbot Pepe
│   └── DogProfiles.kt        # Dog, DogValidator, DogOps, DogStats
└── ui/
    ├── CalculatorScreen.kt   # calculadora + guardar en Mis perros
    ├── MyDogsScreen.kt       # lista, indicadores, formulario
    ├── DogsViewModel.kt  ChatScreen.kt  ChatViewModel.kt  Theme.kt
```

4. Modelo de datos

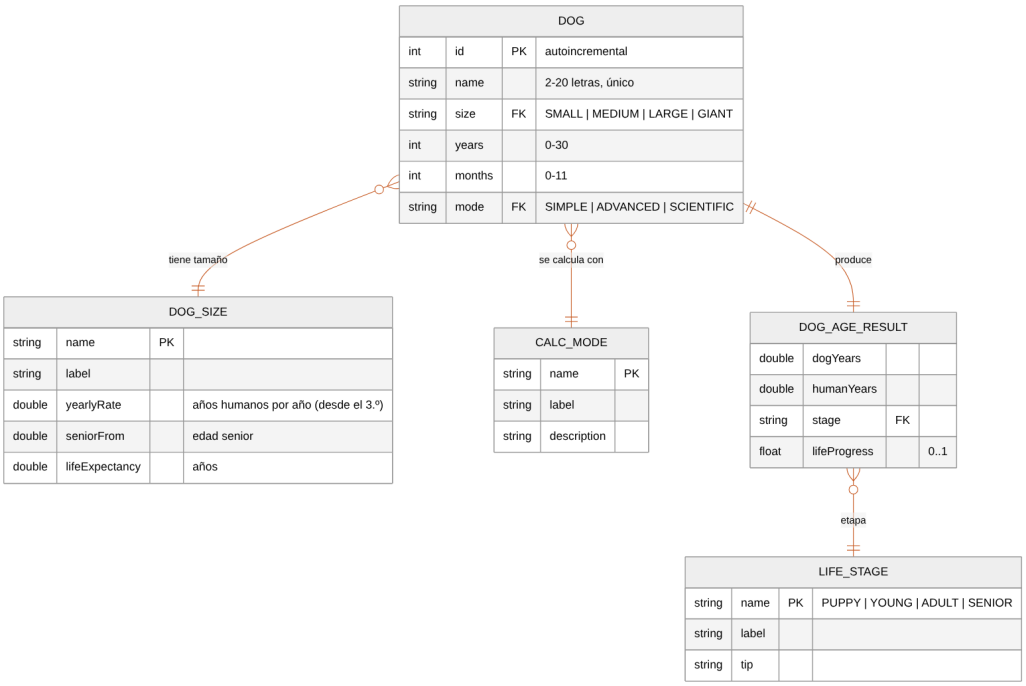


Figura 2. Modelo de datos

5. Clases

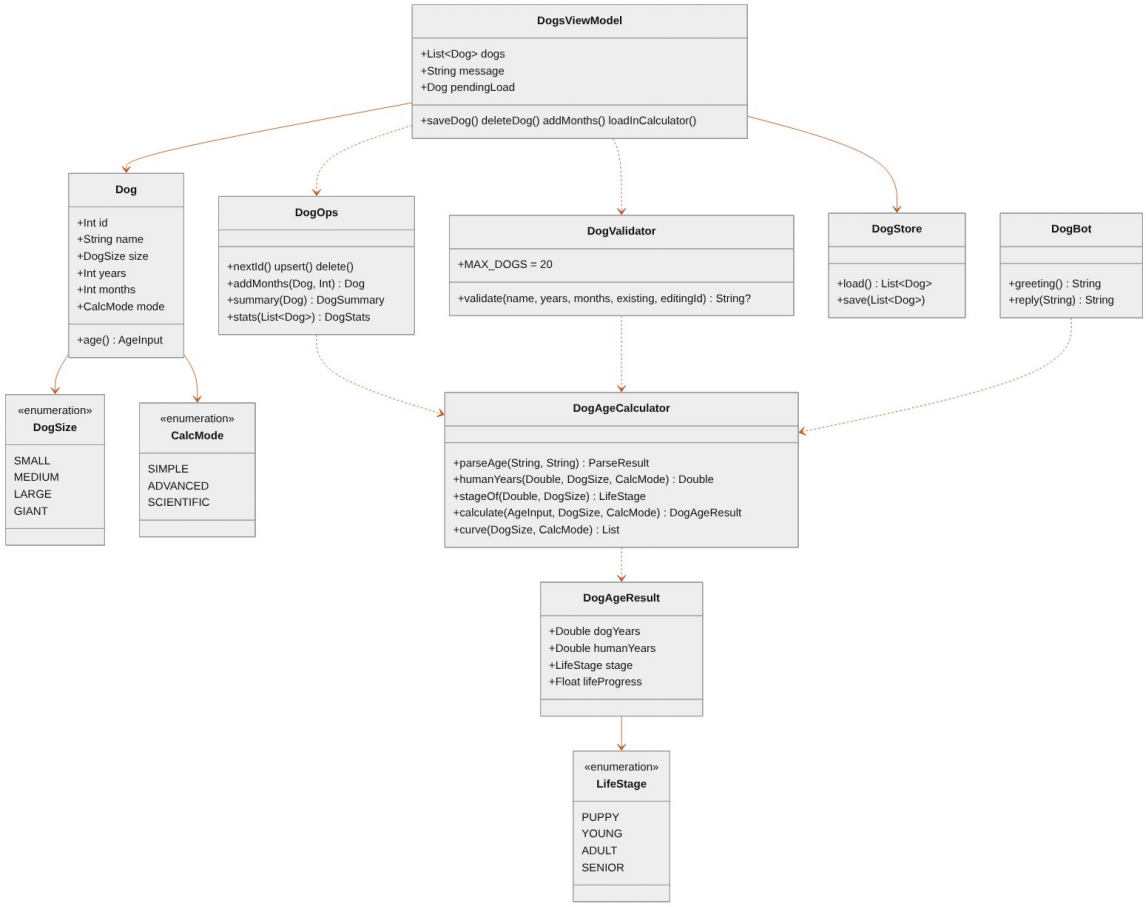


Figura 3. Diagrama de clases

6. Flujos

6.1 Guardar un perro

1. DogFormDialog entrega nombre, tamaño, años, meses y método.
2. DogsViewModel.saveDog llama a DogValidator.validate (nombre, duplicados, límite y DogAgeCalculator.parseAge).
3. Si es válido, crea Dog con DogOps.nextId y aplica DogOps.upsert.
4. Ordena con DogOps.sorted y guarda con DogStore.save.

6.2 Abrir un perro en la calculadora

1. MyDogsScreen llama a vm.loadInCalculator(dog) y cambia a la pestaña 0.
2. CalculatorScreen observa pendingLoad en un LaunchedEffect, copia los datos y llama a consumeLoad().

7. Decisiones de diseño

ID	Decisión	Justificación
DD-01	Persistencia con SharedPreferences + JSON.	Pocos datos (≤ 20 perros); sin dependencias nuevas (Room) y funciona sin internet.
DD-02	DogsViewModel compartido entre Calculadora y Mis perros.	Permite «Calcular» y «Guardar» entre pestañas sin navegación compleja.
DD-03	Validación reutiliza DogAgeCalculator.parseAge.	Mismas reglas de edad en la calculadora y en Mis perros.
DD-04	Edad guardada como años y meses (no fecha de nacimiento).	Coincide con el formulario existente; «+1 mes/+1 año» permite actualizarla.
DD-05	Lógica pura en objetos Kotlin.	26 pruebas JUnit rápidas.